The Answer Stops at Eight: Termination, Not Position Range, Drives Length-Generalisation Failure

Anonymous submission

Abstract

A decoder trained on sequences of length eight fails on sequences of length sixteen, even when the task is copying. That failure is usually attributed to positional encodings (PEs): explicit codes place the model at positions it never saw, and removing them is reported to extrapolate better. We show that this comparison is confounded by a different decision—where to emit the end-of-sequence token. With an explicit code the model learns to terminate at exactly the training length, with EOS probability 1.00 at that position, so every longer answer is truncated there; sequence exact match then reports 0.00 for a model that computes almost everything correctly. Under a forced output length the same checkpoints recover 0.91–0.99 token accuracy, and rotary reaches 1.00. Removing EOS supervision altogether reverses the ranking: rotary copy is perfect at L=16, twice the training length, while NoPE degrades to 0.43 token accuracy. The collapse is therefore not about position ranges—a protocol in which all positions occur in training does not prevent it—and not about where the code enters the network: an absolute code injected at q/k behaves identically to one in the residual stream. It is about addressability of the answer boundary. Exact match ranks PEs partly by termination policy, and we recommend reporting forced-length accuracy beside it.

1 Introduction

A decoder trained on sequences of length eight fails on sequences of length sixteen, even when the task is copying. Whether to include a positional encoding (PE) is the first thing one changes in response, and the decision is fixed before pretraining: reversing it later means retraining. So "which PE" is not a stylistic preference but an expensive commitment, and the evidence behind the current advice is worth getting right.

That advice is that explicit codes hurt and removing them helps. An explicit code assigns absolute positions, a longer sequence puts the model at positions it never saw, and an embedding trained on 0..7 is out of distribution at position 12; on this account a no-PE decoder should extrapolate better, and across ten tasks it does [6]. A second literature treats the same diagnosis with better codes—log-decaying biases [2], resolution-maximising relative codes [17], randomised position sampling [14].

Both rely on a metric that cannot support the conclusion. Exact match is a conjunction over output tokens, so it scores a model that computes nine of ten tokens correctly the same as one that has collapsed—and the same as one that simply stopped after eight. That third case has its own literature: predicting an end-of-sequence (EOS) token changes how models extrapolate, because generation terminates early and hidden states settle into length-indexed attractors [9]. That work varies the EOS decision; the PE comparison varies the code. The two have not been put together, and when they are, the PE ranking on copy does not survive.

What we find. On a copy task, at L_{tr}=8:

- **The failure is a stop.** With sin, rotary, or learned position, the model is correct at every output position up to the eighth and wrong at every position after it, for every test length; its EOS probability is 1.00 at the ninth. Forced to produce ten tokens, it scores 0.93 (sin) and 1.00 (rotary) token accuracy where exact match is 0.00 (Section 5.2).
- **Position ranges are irrelevant.** Fixing the total token count at 41 and padding the tail means all 41 positions occur in every training example. The collapse is unchanged (Section 5.1).

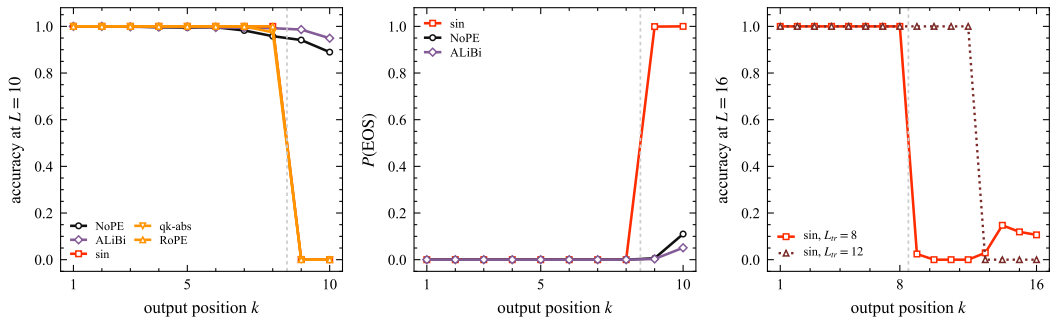


Figure 1: The failure is a stop. (a) Accuracy at each output position, copy, test length 10, L_{tr}=8. Sin, q/k-absolute and rotary are correct at every position up to the eighth and wrong from the ninth; NoPE and ALiBi decay gently. The vertical line marks L_{tr}. (b) Probability assigned to EOS at the same positions: 1.00 from the ninth for sin, below 0.1 throughout for NoPE and ALiBi. (c) The boundary follows the training length: with L_{tr}=12 the same code is correct to the twelfth position at L=16, and at L=8 it truncates at the eighth. Means over two seeds except (c)'s L_{tr}=12 run, which is a single run.

- **Neither the site nor absoluteness explains it.** An absolute code injected at q/k truncates exactly like one added to the residual stream, and rotary—relative by construction—truncates too. What separates the codes is whether the model can address the answer boundary (Section 5.3).
- **Remove the boundary and the ranking reverses.** Trained without any EOS token, rotary copy is perfect at L=16 (1.00 exact match, two seeds) while NoPE falls to 0.43 token accuracy (Section 5.5). The code that "cannot extrapolate" extrapolates furthest once it is not asked to stop.

What follows. The practical consequence is a measurement one: a PE comparison by exact match measures termination policy as much as computation, and reporting forced-length accuracy beside it costs nothing at evaluation time. The scientific consequence is narrower than it looks—we study copying, small models and two seeds—but it is directional: the diagnosis "explicit codes put the model out of distribution" is not what the data show. What the data show is that an addressable boundary gets used.

2 Related Work

Length generalisation and positional encodings. Transformers fit short sequences and degrade sharply beyond them [1, 3]. The explanations offered are mostly computational: attention of fixed depth cannot represent certain length-dependent functions [4]; a task extrapolates when it admits a short program over attention primitives [20, 22]; shallow models learn shortcuts that fit the training lengths and break outside them [7]. Orthogonal to this is the choice of positional code—a distinct vector per position [18], or pairwise information injected through learned relative representations [15], bucketed biases [13], rotations [16] or a linear distance penalty [12]—together with a family of schemes built for the extrapolation regime: log-decaying biases [2], resolution-maximising codes [17], randomised position sampling [14], and priming with long examples [5]; Zhao et al. [21] survey the family. The headline result is that no positional encoding beats five explicit alternatives across ten tasks [6], with follow-up work bounding how far that holds and tracing the residual breakdown to unfocused attention [19]. Almost all of it is scored by exact match.

The EOS decision. A separate literature studies the decision to predict an end-of-sequence token at all. Newman et al. [9] show that models trained to predict EOS extrapolate substantially worse than models that are not, trace the difference to hidden states that stratify by position and then stall, and evaluate in the oracle-length setting we adopt in Section 5.2. Their experiments vary the EOS decision and hold the architecture fixed; ours varies the positional encoding and holds the EOS decision fixed, then removes it—which is what connects the two literatures. Small algorithmic models are the usual place to look for mechanism—grokking as a transition from memorisation to a

structured circuit [8, 11], induction heads as a copying mechanism [10]—and we offer behaviour of that kind rather than a circuit analysis: the profile in Figure 1 constrains what such an account must explain, but does not localise the computation.

3 Where Position Can Enter, and What a Boundary Is

Every encoding we compare is a special case of a single causal attention layer,

$$\mathrm{Attn}(Q,K,V)_i = \sum_{j\le i} \frac{\exp\big(\langle q_i, k_j\rangle/\sqrt{d_h} + B_{ij}\big)}{\sum_{j'\le i}\exp\big(\langle q_i, k_{j'}\rangle/\sqrt{d_h} + B_{ij'}\big)}\, v_j, \tag{1}$$

with $q_i = W_Q h_i$, $k_j = W_K h_j$, $v_j = W_V h_j$ and h_i the residual stream at position i. Position can enter at exactly three sites:

(i) the token representation, $h_i = e(x_i) + p_i$, with p_i learned or a fixed sinusoid (learned, sin);

(ii) the query/key geometry, $q_i \mapsto R_{\Theta,i} q_i$ or $q_i \mapsto q_i + W_q p_i$ (rotary, q/k-absolute);

(iii) the logits, through a bias B_{ij} that carries no token content (ALiBi);

and NoPE is the fourth case, in which all three are left untouched and order is conveyed only by the causal mask $j \le i$.

The logits site cannot host an absolute code. This removes one cell from the design and explains a gap in the literature. Suppose a bias decomposes as $B_{ij} = c_i + B'_{ij}$, with a component that depends on the query index alone. For fixed i, c_i appears in every term of both the numerator and the denominator of (1) and cancels:

$$\frac{e^{c_i} e^{s_{ij}}}{\sum_{j'} e^{c_i} e^{s_{ij'}}} = \frac{e^{s_{ij}}}{\sum_{j'} e^{s_{ij'}}}, \qquad s_{ij} = \langle q_i, k_j\rangle/\sqrt{d_h}. \tag{2}$$

An additive position signal that is constant along the key axis is therefore unobservable. Any logits code that has an effect must vary with j as well as i; it is a function of the *pair*. This is why every effective logits code in the literature—ALiBi, T5 buckets, relative bias generally—is relative, and why "absolute at the logits" is not a condition one can run. It also means the usual battery confounds site with kind: all absolute codes sit at site (i), and the only codes at sites (ii) and (iii) are relative. The q/k-absolute condition of (6) exists to break that confound.

Boundaries. Write $y_1 \dots y_L$ for the answer segment and $\mathrm{stop}(x)$ for the index at which a trained model emits EOS when run autoregressively. We say the model *truncates* when $\mathrm{stop}(x) < L$: it produces a prefix and then declines to continue, rather than producing L tokens of which some are wrong. These are different failures and exact match reports them identically.

A code is *addressable* if, at generation time, the model can compute a quantity that selects a particular output index independently of the token content at that index—an absolute index, or an absolute rotation applied at a known index. NoPE cannot: the only positional information is the causal mask, which orders positions without labelling them. ALiBi cannot: its bias is a function of $i - j$, which orders output positions by distance without naming any of them. The distinction is not absolute-versus-relative—rotary is relative and addressable—and not which site the code occupies, since (6) shows an absolute code at site (ii) behaving like one at site (i). It is about what the model can refer to, and Section 5.3 tests whether it predicts which codes truncate.

4 Experimental Setup

Tasks. All five tasks map a sequence of L symbols from a 10-symbol alphabet to an output sequence of the same length, formatted as $\mathtt{x}_{1..\mathtt{L}}$ <SEP> $\mathtt{y}_{1..\mathtt{L}}$ <EOS>. Cross-entropy is applied only to the output segment. Training uses $L_{\mathrm{tr}} = 8$; unless stated otherwise evaluation uses $L \in \{8, 10, 12, 16, 20\}$, so L=8 measures in-distribution fit and the rest measure extrapolation.

Two protocols. Under the *standard* protocol the sequence has length $2L+2$, so the absolute position of <SEP> is L and the answer occupies positions $L{+}1$ to $2L$. "The model runs out of trained positions" and "the model runs out of trained lengths" are then the same event, which makes the standard protocol unable to distinguish the account we test from the one we reject. Under the *fixed-length* protocol we hold the total token count at $T{=}41$, pin <SEP> at index $K{=}21$, and fill both sides with padding:

$$\underbrace{x_1 \dots x_L}_{\text{input}}\ \underbrace{\varnothing^{K-L}}_{\text{pad}}\ \texttt{<SEP>}\ \underbrace{y_1 \dots y_L}_{\text{answer}}\ \texttt{<EOS>}\ \underbrace{\varnothing^{T-K-L-2}}_{\text{pad}}, \tag{3}$$

with $T \gg L_{\max}$. Every one of the 41 positions therefore occurs in every training example; only the number of *effective* tokens varies. If the collapse survives this—if it appears at $L_{\text{tr}}{+}2$ when no position is new—then it cannot be an out-of-distribution position effect.

Positional encodings. All conditions are instantiations of (1). NoPE sets $p_i = 0$, $B_{ij} = 0$ and applies no rotation, so order is conveyed only by the causal mask. *learned* absolute embeddings use $h_i = e(x_i) + p_i$ with $p_i \in \mathbb{R}^d$ trained, one vector per position. *sin* uses the fixed sinusoid of Vaswani et al. [18],

$$p_{i,2k} = \sin(i\,\omega_k), \qquad p_{i,2k+1} = \cos(i\,\omega_k), \qquad \omega_k = 10000^{-2k/d}. \tag{4}$$

rope rotates queries and keys within each head, $q_i \mapsto R_{\Theta,i}\,q_i$ with $R_{\Theta,i}$ block-diagonal in 2×2 rotations of angle $i\theta_k$ and $\theta_k = 10000^{-2k/d}$, so that $\langle q_i, k_j\rangle$ depends on $i-j$ alone [16]. *alibi* leaves the residual stream untouched and sets

$$B_{ij} = -m_h\,(i-j), \qquad m_h = 2^{-8h/H}, \tag{5}$$

for head $h \in \{1,\dots,H\}$ [12]. Finally *qk-abs* adds the *same* fixed sinusoid of (4), linearly projected, to queries and keys rather than to the residual stream:

$$q_i \leftarrow q_i + W_q p_i, \qquad k_j \leftarrow k_j + W_k p_j. \tag{6}$$

This condition exists to separate two variables that the usual battery confounds: every absolute code in the literature enters at the representation, and the only codes at q/k or at the logits are relative ones. *qk-abs* is an absolute code at a pairwise site, so a difference between it and *sin* is a site effect, and a difference between it and *rope* is an absoluteness effect. Four of the five original conditions—NoPE, learned absolute, rotary and ALiBi—appear in the comparison of Kazemnejad et al. [6]. We substitute a fixed sinusoidal code for their T5-style bucketed relative bias [13], so our battery overlaps with theirs rather than reproducing it; we record the substitution explicitly because it limits how directly the numbers can be compared across the two studies.

Model and optimisation. A decoder-only Transformer with 4 layers, $d_{\text{model}}{=}256$, 8 heads, pre-layered-norm blocks, GELU MLP with hidden width $4d$, and a tied output head: about 3.2M parameters. AdamW with learning rate 5×10^{-4}, $\beta = (0.9, 0.98)$, weight decay 0.01, batch size 128, 2000 steps, 100-step linear warmup then cosine decay, gradients clipped to norm 1.0. Each step draws a fresh batch rather than sampling a fixed finite training set, so every condition sees the same number of i.i.d. examples and memorisation of a particular sample cannot explain a difference.

Four measurements. Exact match is a conjunction over L tokens, so it reports a stop and a computation as the same zero. We therefore report, for each condition and length,

1. Acc_L: sequence exact match, the standard metric;
2. Tok_L: per-token accuracy over the answer segment, which separates a wrong token from a missing one;
3. $\text{Acc}_L^{\text{ora}}$: exact match with the EOS logit masked at every answer position, so generation cannot terminate early and the model is forced to produce L tokens. This is the oracle-length evaluation used by Newman et al. [9], and it measures content alone;

Table 1: The result we set out to explain: extrapolation score (mean exact match over $L \in \{10, 12, 16, 20\}$, two seeds) on the standard protocol, where every code but NoPE and ALiBi scores zero on four of five tasks. Reproduces the ordering reported by Kazemnejad et al. [6]. The rest of this section shows that most of this table is a termination artefact.

Task	NoPE	learned	sin	RoPE	ALiBi
copy	**0.34**	0.00	0.00	0.00	0.31
prefix-XOR	0.00	0.00	0.00	0.00	0.00
sort	**0.76**	0.00	0.00	0.00	0.00
swap-halves	0.00	0.00	0.00	0.00	0.00
reverse	**0.29**	0.00	0.00	0.00	0.00

4. stop_L: the mean index of the first position predicted as EOS, with L recorded when no EOS is predicted.

A model that stops at $\text{stop}_L = L_{\text{tr}} < L$ truncates; a model with $\text{stop}_L \approx L$ terminates correctly and fails, if at all, on content.

The solved control. A condition counts as *solved* if in-distribution accuracy at L=8 reaches 0.95. Extrapolation failure is only interpretable for models that learned the task; without this control a code can appear to "fail to extrapolate" when it failed to fit. Unsolved conditions are reported rather than dropped.

Seeds and standard errors. Each cell is run with 2 seeds and every figure reports the mean with the seed spread; every quoted number is the mean over seeds. With two seeds the design is underpowered for fine pairwise claims, so we restrict ourselves to differences that are an order of magnitude apart—which, as Section 5 shows, is the scale of the effects here: the quantities we contrast are 0.00 against 0.91 and 8 against 10, not 0.41 against 0.44.

5 Results

Table 1 is the phenomenon. On five tasks and five encodings, under the standard protocol, NoPE is best or tied everywhere and every explicit code scores zero on four of the five tasks. Two of those zeros—*prefix-XOR* and *swap-halves*—are failures to fit as much as to extrapolate and are excluded from the analysis below; the interesting cases are *copy*, *sort* and *reverse*, where the model reaches in-distribution accuracy 1.00. On copy, NoPE scores 0.34 and every explicit code 0.00. That gap is what the rest of this section decomposes.

5.1 Positions are not the variable

We first run the experiment the position-range account predicts should succeed. Under the fixed-length protocol of (3), every one of the 41 positions occurs in every training example, so no test token sits at a position the model has not seen. If the collapse is caused by out-of-distribution positions, it should disappear.

It does not. Table 2, row L=10: learned, sinusoidal, rotary and q/k-absolute codes all reach exact match 1.000 at L=8 and 0.000 at L=10, a two-token increase in effective length with every position in range. NoPE and ALiBi, run under the identical protocol, retain 0.78 and 0.92. The position-range account therefore cannot be the whole story; something changes between L=8 and L=10 that is not the set of positions.

5.2 The failure is a stop, not a computation

Exact match is a conjunction over L tokens, so it cannot say whether the model computed eight tokens and stopped or computed ten and got them wrong. Per-output-position accuracy separates the two, and the profile is unmistakable (Figure 1a). For the four truncating codes accuracy is 1.00 at every output position up to the eighth and exactly 0.00 from the ninth; NoPE and ALiBi decay gently instead. At L=10 this is not a subtle trend: token accuracy is 0.800 for sin, 0.797 for q/k-absolute and 0.800 for rotary, which is eight correct tokens out of ten, to three decimals.

Table 2: The fixed-length protocol on copy, mean over two seeds. In every training example all 41 positions occur, so no test position is out of range; only the effective length changes. Acc is sequence exact match, Tok is per-token accuracy over the answer, Acc^{ora} forces the correct length by masking EOS, and *stop* is the mean index of the first predicted EOS. Every code with an addressable boundary stops at exactly L_{tr}=8 at every test length; its token accuracy at L=10 is then 0.80—eight tokens out of ten—while exact match reads 0.00.

	$L = 10$				$L = 12$			
Encoding	Acc	Tok	Acc^{ora}	stop	Acc	Tok	Acc^{ora}	stop
NoPE	**0.78**	**0.98**	0.99	9.9	**0.03**	**0.79**	0.88	10.7
ALiBi	0.92	0.99	**1.00**	9.9	0.14	0.87	**0.94**	10.8
learned	0.00	0.81	0.83	8.0	0.00	0.69	0.72	8.0
sin	0.00	0.80	0.93	8.0	0.00	0.67	0.79	8.0
q/k-abs	0.00	0.80	0.91	8.0	0.00	0.66	0.77	8.0
RoPE	0.00	0.80	1.00	8.0	0.00	0.67	0.98	8.0

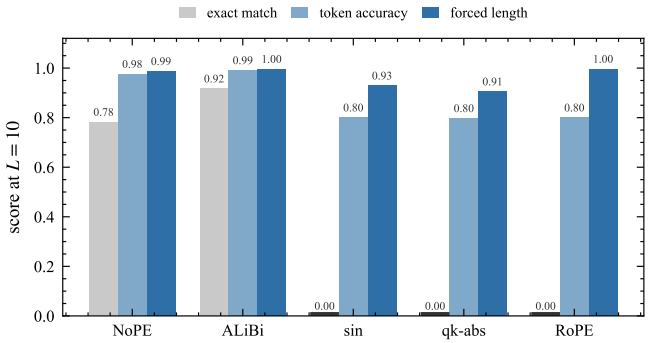


Figure 2: Exact match, token accuracy and forced-length accuracy at L=10, copy. The truncating codes score 0.00 exact match on content that is 80% correct, and recover to 0.91–1.00 once generation cannot terminate early. Black marks at zero indicate a measured zero.

The model is not failing to compute the ninth token; it is declining to emit it. Figure 1b shows the EOS probability at each output position. For sin it is below 0.01 for the first eight positions and 1.00 at the ninth, where it stays; NoPE and ALiBi never exceed 0.1 there. The mean stop position (Table 2, last column of each block) is 8.0 for all four truncating codes at every test length, and 9.9 and 10.7 for NoPE at L=10 and 12.

Forcing the length confirms it (Figure 2). With the EOS logit masked at every answer position, the same checkpoints score 0.93 (sin), 0.91 (q/k-absolute) and 1.00 (rotary) token accuracy at L=10, against exact match 0.00 for all three. Rotary is the extreme case: forced-length accuracy 0.996 at L=10 and 0.984 at L=12, indistinguishable from NoPE, while exact match is 0.00 at both. For that code the entire sequence-level failure is the stop decision.

Two caveats belong here rather than in a limitations section. First, content is not untouched for the additive codes: forced-length accuracy is 0.93 for sin, 0.91 for q/k-absolute and 0.83 for learned, against 0.99 for NoPE, so additive position codes cost between six and sixteen points of token accuracy at L=10 on top of the truncation. Termination dominates the sequence-level score but does not explain all of it. Second, at L=16 rotary's stop behaviour becomes erratic—mean stop position 2.2 across the two seeds, with the runs disagreeing—so we make no claims about rotary beyond L=12 in this protocol.

5.3 Which codes truncate

The truncating set is $\{\text{learned}, \text{sin}, \text{rotary}, q/k\text{-absolute}\}$, the non-truncating set $\{\text{NoPE}, \text{ALiBi}\}$. Two explanations available in the literature are ruled out by this partition.

Not the injection site. Every absolute code in the standard battery enters at the residual stream, and the only codes at q/k or at the logits are relative, so site and absoluteness are perfectly con-

founded there. The q/k-absolute condition breaks the confound: the same fixed sinusoid, linearly projected into queries and keys instead of added to representations. It truncates in exactly the same place (stop=8.0; token accuracy 0.797 versus 0.800; forced-length 0.905 versus 0.929). Moving an absolute code between sites changes nothing measurable.

Not absoluteness, and not relativity either. Rotary is relative by construction—$\langle q_i, k_j \rangle$ depends on $i - j$ alone—and it truncates. Relative coding is therefore not sufficient for extrapolation, contrary to the usual summary of the relative-versus-absolute literature, and absoluteness is not necessary for truncation. What the two sets share is a property of the *boundary* rather than of the code's geometry: sin, learned, rotary and q/k-absolute each let the model name an output position exactly—by an index, or by an absolute rotation applied at a known index—so the end of the answer is an address it can look up. NoPE can name no position, and ALiBi can only compare distances, which order output positions without labelling them; neither can place a boundary at a chosen index, and neither truncates.

5.4 The boundary follows the training length

If the stop is addressed rather than computed, it should move when the training length moves. Training sin with L_{tr}=12 gives prefix length 12 at L=14, 16 and 18—stop position 12 at each—while L=10 and 12 are exactly correct, being in distribution (Figure 1c). The boundary tracks L_{tr}, not any property of the position range, and its length equals the answer segment the model was trained to produce.

Training on several lengths sharpens the contrast rather than removing it. With $L_{\text{tr}} \in \{6, 8, 10\}$ sampled uniformly, sinusoidal copy is perfect at 4, 6, 8 and 10, and 0.00 at L=12, the first unseen length, in both seeds. NoPE under the identical schedule reaches 0.99 at L=12 and 0.42 at L=16. A mixture over lengths supplies the absolute code with one boundary per length—it does not teach where to stop—while a model with no addressable boundary simply extrapolates through the unseen length. The difference between the two codes is therefore not a difference in training data, which is identical here, but in whether a stop can be encoded as a constant.

5.5 Removing the boundary reverses the ranking

The stop account makes a causal prediction: remove the EOS token from training altogether, so that every target is a content token and no boundary is ever placed, and the codes should converge. They do—and past that. Figure 3 reports the result. Under no-EOS training, rotary copy is perfect at every length we test: 1.00 exact match at L=8, 10, 12 and 16, in both seeds. NoPE under the same training reaches 0.94 at L=10, 0.13 at L=12 and 0.00 at L=16, with token accuracy 0.43 there. Sinusoidal sits between, at 0.54 at L=10 with token accuracy 0.96.

So the ranking that motivates the no-PE advice reverses when the termination decision is removed: the code that "cannot extrapolate" extrapolates furthest, and the code that is recommended for extrapolation is the one that degrades. We do not claim rotary is generally better than NoPE—we test copying, at one model size, with two seeds—but the direction of this result is not ambiguous, and it is not what an out-of-distribution-position story predicts.

6 Discussion

What the comparison was measuring. The ordering in Table 1 is reproducible and, as a description of trained models under the usual recipe, correct. What it does not license is the inference that explicit codes compute worse. On copy the opposite holds: under a forced output length rotary reaches 0.996 token accuracy at L=10 and 0.984 at L=12 against 0.987 and 0.877 for NoPE, and with EOS removed from training rotary is perfect at L=16 while NoPE falls to 0.43. The standard protocol was measuring where models stop.

Why the boundary lands on L_{tr}. A model trained at one length sees exactly L_{tr} answer tokens in every example. If it can address output positions, the rule "emit EOS at index $L_{\text{tr}}+1$" is a single parameter that fits the training distribution perfectly and is simply undefined outside it; nothing in the objective penalises that. If it cannot address positions, the same rule is not representable, and the model must fall back on content or on a distance cue, both of which are fuzzier and later—

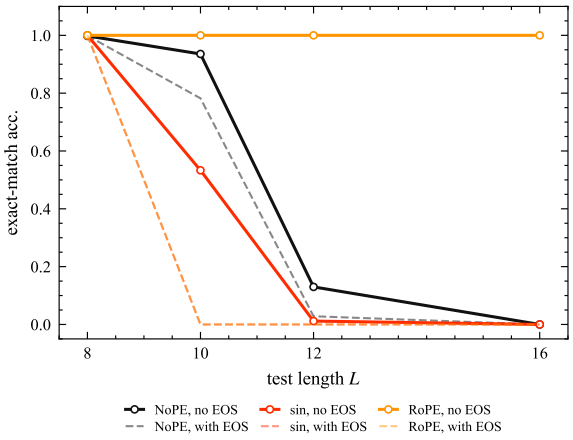


Figure 3: Exact match with EOS supervision (dashed) and without it (solid), copy, mean of two seeds. Removing the boundary lifts every code and reverses the order: rotary reaches 1.00 at L=16, twice the training length, while NoPE falls to 0.00. The no-EOS models are evaluated at a fixed output length by construction, which is the point of the comparison.

which is what the NoPE and ALiBi profiles in Figure 1 show. The asymmetry is not about the informativeness of the code but about whether a stop can be encoded as a constant.

This is the mechanism behind the "length attractor" of Newman et al. [9]: they showed that models trained to predict EOS stratify hidden states by position and stall there, and we locate the consequence at the level of a cross-code comparison. It is also why a mixture over lengths does not help the absolute code (Section 5.4): the constant becomes a lookup with one entry per length, but it is still a lookup, and the entry for the test length does not exist.

Relation to the no-PE recommendation. Kazemnejad et al. [6] compare five encodings on ten tasks and find NoPE best. Our Table 1 reproduces that ordering on our tasks and our model size. The result here does not overturn their measurement; it reinterprets what it measures. On tasks where the answer has the same length as the input, an exact-match comparison is partly a comparison of how crisply each code can place a stop. The practical corollary is cheap: report forced-length accuracy alongside exact match, or evaluate with EOS masked, and the comparison becomes about computation again.

Why ALiBi is the exception in the standard table. ALiBi is the only standard code that both leaves the residual stream alone and is unaddressable: its bias orders output positions by distance from the query without naming any of them. It therefore stops late and softly rather than at a constant, which is what its 0.92 exact match at L=10 reflects. We would predict that any code with a monotone and unaddressable positional signal behaves similarly, and that ALiBi's advantage would shrink under no-EOS training—a prediction we did not test.

What this does not show. We study copying with a 3.2M-parameter model, two seeds, and one training length per condition. *sort*, *reverse* and *prefix-XOR* are in Table 1 but not in the decomposition, and the permutation tasks may not decompose the same way, since a truncated answer there is wrong at every position rather than at the tail. We also did not test whether the effect survives scale, pretraining, or natural language, where answers rarely have the same length as inputs and the boundary is cued by content rather than by a count. The claim we are prepared to defend is the narrow one: on copy-like same-length tasks, at this scale, the cross-code ranking on exact match is dominated by termination, and it reverses when termination is removed.

7 Limitations

One task for the strongest claims. The decomposition, the q/k-absolute control and the no-EOS experiment are all run on copy. Copy is the cleanest case—the answer has the same length as the input, so a truncated answer is correct exactly on its prefix—but it is also the most favourable case for our argument, and the permutation tasks may not decompose the same way. Extending the per-position analysis to *reverse* and *sort* is the first thing we would do next; we report only their exact-match behaviour here.

Scale and distribution. Our models have 3.2M parameters and are trained from scratch on synthetic sequences, two seeds per condition. Nothing in the mechanism requires scale, but nothing here establishes that it survives pretraining, larger models, or natural language, where answers rarely match input lengths and the end of an answer is cued by content rather than by a count. The claim we defend is correspondingly narrow.

The no-EOS condition changes the task. Removing the EOS token removes the need to decide when to stop, so the model is evaluated at a fixed length by construction. That is the point of the comparison—it isolates content—but it also means the no-EOS numbers are not an alternative training recipe for generation; a model that never predicts EOS cannot stop.

Two seeds. Enough to see that a 1.00-versus-0.00 gap is not noise, not enough for fine ranking claims. The no-EOS condition shows this directly: sinusoidal copy lands at 0.28 and 0.79 across two seeds, so we report it as confirming a mechanism rather than as a fix. Every number quoted is a mean over two seeds and we restrict claims to differences an order of magnitude apart.

Online sampling. Each step draws a fresh batch rather than sampling from a fixed finite training set. This removes memorisation of a particular sample as an explanation for a difference between codes and equalises the number of examples seen, but it makes the task easier than the fixed-train-set protocol of some prior work, so absolute accuracies are not comparable across studies.

What we did not test. We did not test whether masking EOS at inference recovers generation quality in a realistic setting, whether length conditioning removes the truncation, or whether the addressability distinction predicts behaviour for codes outside our battery—monotone biases other than ALiBi, learned relative codes, or codes at several sites at once. The ALiBi prediction in Section 6 is stated as a prediction for exactly this reason.

8 Conclusion

The question we began with was why explicit positional encodings fail to extrapolate, given that the usual explanation—positions outside the trained range—is easy to state and easy to check. It is not the explanation. When every position occurs in every training example the failure is unchanged, and when the answer is examined position by position it turns out not to be a failure of computation at all: the model computes the whole answer and emits a stop token after L_{tr} tokens, with probability 1.00, at every test length.

That reframes a comparison the field has been reading as a statement about encodings. Exact match ranks positional encodings partly by how well each can place a stop, and removing the stop reverses the ranking: the code usually reported as worst for extrapolation is the only one that extrapolates perfectly once generation cannot terminate early. The mechanism is narrow and checkable—an addressable boundary gets placed at the training length, because that is the cheapest rule that fits—and it suggests that when an encoding is blamed for a length failure, the first thing to measure is where generation stopped.

References

[1] Cem Anil, Yuhuai Wu, Anders Andreassen, Aitor Lewkowycz, Vedant Misra, Vinay Ramasesh, Ambrose Slone, Guy Gur-Ari, Ethan Dyer, and Behnam Neyshabur. Exploring length

generalization in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 38546–38556, 2022.

[2] Ta-Chung Chi, Ting-Han Fan, Alexander Rudnicky, and Peter Ramadge. Dissecting transformer length extrapolation via the lens of receptive field analysis. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13522–13537, 2023.

[3] Grégoire Delétang, Anian Ruoss, Jordi Grau-Moya, Tim Genewein, Li Kevin Wenliang, Elliot Catt, Chris Cundy, Marcus Hutter, Shane Legg, Joel Veness, and Pedro A. Ortega. Neural networks and the chomsky hierarchy. In *International Conference on Learning Representations (ICLR)*, 2023.

[4] Michael Hahn. Theoretical limitations of self-attention in neural sequence models. *Transactions of the Association for Computational Linguistics*, 8:156–171, 2020.

[5] Samy Jelassi, Stéphane d'Ascoli, Carles Domingo-Enrich, Yuhuai Wu, Yuanzhi Li, and François Charton. Length generalization in arithmetic transformers. *arXiv preprint arXiv:2306.15400*, 2023.

[6] Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[7] Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers learn shortcuts to automata. In *International Conference on Learning Representations (ICLR)*, 2023.

[8] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations (ICLR)*, 2023.

[9] Benjamin Newman, John Hewitt, Percy Liang, and Christopher D. Manning. The EOS decision and length extrapolation. In *Proceedings of the Workshop on BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP (BlackboxNLP)*, 2020. arXiv:2010.07174.

[10] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. Transformer Circuits Thread, 2022. arXiv:2209.11895.

[11] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

[12] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations (ICLR)*, 2022.

[13] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 2020.

[14] Anian Ruoss, Grégoire Delétang, Tim Genewein, Jordi Grau-Moya, Róbert Csordás, Mehdi Bennani, Shane Legg, and Joel Veness. Randomized positional encodings boost length generalization of transformers. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1889–1903, 2023.

[15] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of NAACL-HLT*, pages 752–761, 2018.

[16] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[17] Yutao Sun, Li Dong, Barun Patra, Shuming Ma, Shaohan Huang, Alon Benhaim, Vishrav Chaudhary, Xia Song, and Furu Wei. A length-extrapolatable transformer. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14590–14604, 2023.

[18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.

[19] Jie Wang, Tao Ji, Yuanbin Wu, Hang Yan, Tao Gui, Qi Zhang, Xuanjing Huang, and Xiaoling Wang. Length generalization of causal transformers without position encoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 14024–14040, 2024.

[20] Gail Weiss, Yoav Goldberg, and Eran Yahav. Thinking like transformers. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139, pages 11080–11090, 2021.

[21] Liang Zhao, Xiachong Feng, Xiaocheng Feng, Weihong Zhong, Dongliang Xu, Qing Yang, Hongtao Liu, Bing Qin, and Ting Liu. Length extrapolation of transformers: A survey from the perspective of positional encoding. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 9959–9977, 2024.

[22] Hattie Zhou, Arwen Bradley, Etai Littwin, Noam Razin, Omid Saremi, Josh Susskind, Samy Bengio, and Preetum Nakkiran. What algorithms can transformers learn? A study in length generalization. In *International Conference on Learning Representations (ICLR)*, 2024.